

DR. APJ ABDUL KALAM TECHNICAL UNIVERSITY (AKTU)
KCC Institute of Technology and Management, Greater Noida

WEEK 2

MODULE

Lab Assignment | PDF Submission
3rd Year | B.Tech CSE

Name: Mahi Agarwal
Roll No.: 2404920100189

Section: A5
Submission: 22nd June 2026

Linear Search

1. Aim:

To search a given element in an array using linear search.

Theory:

Linear Search is a simple searching technique in which each element of the array is checked one by one until the required element is found. It can be used on both sorted and unsorted arrays. If the element is present, its position is returned, otherwise, the search ends after checking all elements.

Algorithm:

1. Start from the first element of the array.
2. Compare the current element with the target element.
3. If both are equal, the element is found.
4. Otherwise move to the next element.
5. Repeat the process until the element is found or the array ends.
6. Display the result.

Complexity:

- Time complexity : $O(n)$
- Space complexity : $O(1)$

Fraud Transaction Detection (Linear Search)

Problem statement:

A bank store transaction IDs in the order they occur. Since the transactions are not sorted, the security team needs to check if a suspicious transaction ID exists.

write a program using linear search.

Input: 5

1201 1305 14050 1408 1502

1450

```
#include <iostream>
```

```
using namespace std;
```

```
int main() {
```

```
    int n, target;
```

```
    cin >> n;
```

```
    int arr[n];
```

```
    for (int i = 0; i < n; i++) {
```

```
        cin >> arr[i];
```

```
    } cin >> target;
```

```
    bool found = false;
```

```
    for (int i = 0; i < n; i++) {
```

```
        if (arr[i] == target) {
```

```
            found = true;
```

```
            break; } }
```

```
    if (found)
```

```
        cout << "Fraud transaction found";
```

```
    else
```

```
        cout << "Transaction Not found";
```

```
    return 0;
```

```
}
```


 **OnlineGDB**

online compiler and debugger for c/c++

code. compile. run. debug. share.

IDE

My Projects

Classroom new

Learn Programming

Programming Questions

Sign Up

Login

Run Debug Stop Share Save Beautify

main.cpp

```
3
4 int main() {
5     int n, target;
6     cin >> n;
7
8     int arr[n];
9     for(int i = 0; i < n; i++) {
10         cin >> arr[i];
11     }
12
13     cin >> target;
14
15     bool found = false;
16
17     for(int i = 0; i < n; i++) {
18         if(arr[i] == target) {
19             found = true;
20             break;
21         }
22     }
23
24     if(found)
25         cout << "Fraud Transaction Found";
26     else
27         cout << "Transaction Not Found";
28
29     return 0;
30 }
```

input

```
5
1201 1305 1450 1408 1502
1450
Fraud Transaction Found

...Program finished with exit code 0
Press ENTER to exit console.
```

Language C++

28

Binary Search

Aim:

To search an element in a sorted array using Binary Search.

Theory:

Binary Search is an efficient searching algorithm that works only on sorted arrays. It repeatedly divides the array into two halves and checks the middle element. This process continues until the required element is found.

Algorithm:

1. Find the middle element of the array.
2. Compare it with the target element.
3. If equal, the element is found.
4. If the target is smaller, search the left half.
5. If the target is greater, search the right half.
6. Repeat until the element is found or the search space becomes empty.

Complexity:

- Time complexity: $O(\log n)$
- Space complexity: $O(1)$

Server log Search (Binary Search)

Problem Statement:

A cloud company stores server log IDs in sorted order. Engineers need to quickly verify if a log ID exists. WAP using Binary Search.

Input:

1001 1005 1010 1015 1020 1030 1040
1015

```
#include <iostream>
```

```
using namespace std;
```

```
int main() {
```

```
    int n, target;
```

```
    cin >> n;
```

```
    int arr[n];
```

```
    for (int i = 0; i < n; i++) {
```

```
        cin >> arr[i];
```

```
    }
    cin >> target;
```

```
    int low = 0, high = n - 1;
```

```
    bool found = false;
```

```
    while (low <= high) {
```

```
        int mid = (low + high) / 2;
```

```
        if (arr[mid] == target) {
```

```
            found = true;
```

```
            break;
        }
```

```
        else if (arr[mid] < target) {
```

```
            low = mid + 1;
```

```
        }
        else {
```

```
            high = mid - 1;
```

```
        }
    }
```

```
    if (found) {
```

```
        cout << "log found";
```

```
    }
    else {
```

```
        cout << "log not found";
```

```
    }
    return 0;
```

OnlineGDB

online compiler and debugger for c/c++

code. compile. run. debug. share.

IDE

My Projects

Classroom new

Learn Programming

Programming Questions

Sign Up

Login

main.cpp

Run

Debug

Stop

Share

Save

Beautify

Language C++

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

31

32

33

34

35

36

37

cin >> arr[1];

}

cin >> target;

int low = 0, high = n - 1;

bool found = false;

while(low <= high) {

int mid = (low + high) / 2;

if(arr[mid] == target) {

found = true;

break;

}

else if(arr[mid] < target)

low = mid + 1;

else

high = mid - 1;

}

if(found)

cout << "Log Found";

else

cout << "Log Not Found";

return 0;

input

7

1001 1005 1010 1015 1020 1030 1040

1015

Log Found

...Program finished with exit code 0

Press ENTER to exit console.

out • FAQ • Docs • Blog • Terms of Use • Contact Us

• GDB Tutorial • Credits • Privacy

ns://www.onlinegdb.com GDB Online

34

Bubble Sort

Aim:

To sort elements of an array using Bubble Sort.

Theory:

Bubble Sort repeatedly compares adjacent elements and swaps them if they are in the wrong order. After each pass, the largest element moves to its correct position. The process continues until the array becomes sorted.

Algorithm:

1. Compare adjacent elements.
2. Swap them if they are in the wrong order.
3. Continue comparison until the end of the array.
4. Repeat the process for all elements.
5. Stop when no more swaps are required.

Complexity:

- Time complexity: $O(n^2)$
- Space complexity: $O(1)$

38 Sort Employee Salaries (Bubble Sort)

A company wants to sort employee salaries in ascending order for payroll analysis.

Input:

5
45000 32000 55000 28000 6000

```
#include <iostream>
```

```
using namespace std;
```

```
int main() {
```

```
    int n;
```

```
    cin >> n;
```

```
    int arr[n];
```

```
    for (int i = 0; i < n; i++)
```

```
        cin >> arr[i];
```

```
    for (int i = 0; i < n - 1; i++) {
```

```
        for (int j = 0; j < n - i - 1; j++) {
```

```
            if (arr[j] > arr[j + 1]) {
```

```
                int temp = arr[j];
```

```
                arr[j] = arr[j + 1];
```

```
                arr[j + 1] = temp;
```

```
            }
```

```
        }
```

```
    }
```

```
    for (int i = 0; i < n; i++)
```

```
        cout << arr[i] << " ";
```

```
    return 0;
```

```
}
```

OnlineGDB

online compiler and debugger for c/c++

code. compile. run. debug. share.

IDE

My Projects

Classroom new

Learn Programming

Programming Questions

Sign Up

Login

onlinegdb.com/online_c++_compiler#

Run Debug Stop Share Save Beautify

Language C++

main.cpp

```
1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     int n;
6     cin >> n;
7
8     int arr[n];
9
10    for(int i = 0; i < n; i++)
11        cin >> arr[i];
12
13    for(int i = 0; i < n - 1; i++) {
14        for(int j = 0; j < n - i - 1; j++) {
15            if(arr[j] > arr[j + 1]) {
16                int temp = arr[j];
17                arr[j] = arr[j + 1];
18                arr[j + 1] = temp;
19            }
20        }
21    }
22
23    for(int i = 0; i < n; i++)
24        cout << arr[i] << " ";
25
26    return 0;
27 }
```

input

5
45000 32000 55000 28000 60000
28000 32000 45000 55000 60000

...Program finished with exit code 0
Press ENTER to exit console.

FAQ Docs Blog Terms of Use Contact Us

GDB Tutorial Credits Privacy

44 Insertion Sort

Aim:

To sort elements of an array using insertion sort.

Theory:

Insertion Sort builds a sorted array one element at a time. Each element is inserted into its correct position among the already sorted elements. It is efficient for small datasets.

Algorithm:

1. Assume the first element is sorted.
2. Select the next element.
3. Compare it with previous elements.
4. Shift larger elements one position to the right.
5. Insert the selected element at the correct position.
6. Repeat until all elements are sorted.

Complexity:

- Time complexity: $O(n^2)$
- Space complexity: $O(1)$

44 Sort website Response Time (Insertion Sort)

Problem Statement :

A web monitoring system records website response time. New data comes continuously and must be inserted into the correct position.

Input : 5

25.0 120 320 180 100

```
#include <iostream>
```

```
using namespace std;
```

```
int main() {
```

```
    int n;
```

```
    cin >> n;
```

```
    int arr[n];
```

```
    for (int i = 0; i < n; i++)
```

```
        cin >> arr[i];
```

```
    for (int i = 1; i < n; i++) {
```

```
        int key = arr[i];
```

```
        int j = i - 1;
```

```
        while (j >= 0 && arr[j] > key) {
```

```
            arr[j+1] = arr[j];
```

```
            j--;
```

```
        }
```

```
        arr[j+1] = key;
```

```
    }
```

```
    for (int i = 0; i < n; i++)
```

```
        cout << arr[i] << " ";
```

```
    return 0;
```

```
}
```

OnlineGDB

online compiler and debugger for c/c++

code. compile. run. debug. share.

IDE

My Projects

Classroom new

Learn Programming

Programming Questions

Sign Up

Login

main.cpp

Run

Debug

Stop

Share

Save

Beautify

Language: C++

```
1 using namespace std;
2
3
4 int main() {
5     int n;
6     cin >> n;
7
8     int arr[n];
9
10    for(int i = 0; i < n; i++)
11        cin >> arr[i];
12
13    for(int i = 1; i < n; i++) {
14        int key = arr[i];
15        int j = i - 1;
16
17        while(j >= 0 && arr[j] > key) {
18            arr[j + 1] = arr[j];
19            j--;
20        }
21
22        arr[j + 1] = key;
23    }
24
25    for(int i = 0; i < n; i++)
26        cout << arr[i] << " ";
27
28    return 0;
29 }
```

input

5
250 120 320 180 100
100 120 180 250 320

...Program finished with exit code 0
Press ENTER to exit console.

about • FAQ • Docs • Blog • Terms of Use • Contact Us

• GDB Tutorial • Credits • Privacy

5. Selection Sort

Aim:

To sort element of an array using Selection Sort.

Theory:

Selection sort repeatedly selects the smallest element from the unsorted portion of the array and places it at the beginning. This process continues until the entire array becomes sorted.

Algorithm :

1. Find the smallest element in the array.
2. Swap it with the first unsorted element.
3. Move the sorted boundary forward.
4. Repeat the process for the remaining elements.
5. Continue until the array is completely sorted.

Complexity :

- Time complexity: $O(n^2)$
- Space complexity: $O(1)$

Problem Statement:

A software company assigns priority scores to bug reports. To resolve bugs efficiently, they want to sort priorities in ascending order.

Input:

5

9 2 8 15

```
#include <iostream>
```

```
using namespace std;
```

```
int main() {
```

```
    int n;
```

```
    cin >> n;
```

```
    int arr[n];
```

```
    for (int i = 0; i < n; i++)
```

```
        cin >> arr[i];
```

```
    for (int i = 0; i < n - 1; i++) {
```

```
        int minIndex = i;
```

```
        for (int j = i + 1; j < n; j++) {
```

```
            if (arr[j] < arr[minIndex])
```

```
                minIndex = j;
```

```
        }
```

```
        int temp = arr[i];
```

```
        arr[i] = arr[minIndex];
```

```
        arr[minIndex] = temp;
```

```
    }
```

```
    for (int i = 0; i < n; i++)
```

```
        cout << arr[i] << " ";
```

```
    return 0;
```

```
}
```

OnlineGDB
online compiler and debugger for c/c++
code, compile, run, debug, share.
IDE
My Projects
Classroom new
Learn Programming
Programming Questions
Sign Up
Login

main.cpp

```
1 int main() {  
2     int n;  
3     cin >> n;  
4  
5     int arr[n];  
6  
7     for(int i = 0; i < n; i++)  
8         cin >> arr[i];  
9  
10    for(int i = 0; i < n - 1; i++) {  
11        int minIndex = i;  
12  
13        for(int j = i + 1; j < n; j++) {  
14            if(arr[j] < arr[minIndex])  
15                minIndex = j;  
16        }  
17  
18        int temp = arr[i];  
19        arr[i] = arr[minIndex];  
20        arr[minIndex] = temp;  
21    }  
22  
23    for(int i = 0; i < n; i++)  
24        cout << arr[i] << " ";  
25  
26    return 0;  
27 }
```

input

```
5  
1 2 5 8 9  
1 2 5 8 9  
...Program finished with exit code 0  
Press ENTER to exit console.
```

Run
Debug
Stop
Share
Save
Beautify

Language: C++

out • FAQ • Docs • Blog • Terms of Use • Contact Us
• GDB Tutorial • Credits • Privacy
© 2016 - 2022 OnlineGDB